

Verificação, Validação e Teste de Software (VVT) 2026.1

Day 4 - Conceitos básicos para a aplicação das atividades de VVT

Prof. Dr. Awdren de Lima Fontão



UNIVERSIDADE
FEDERAL DE
MATO GROSSO DO SUL



Verificação, Validação & Teste (VVT)



"A principal causa da crise do software é que as máquinas se tornaram várias ordens de magnitude **mais poderosas!** Falando sem rodeios: enquanto não houvesse máquinas, a programação não constituía qualquer problema; quando tínhamos alguns computadores fracos, a programação tornou-se um problema moderado, e agora temos computadores gigantescos, **a programação tornou-se um problema igualmente gigantesco.**"

Edsger Dijkstra, *The Humble Programmer*

Principais dificuldades na construção de software

segundo Brooks

"Nossa sociedade funciona à base de software"

B. Stroustrup



Complexidade



Adaptabilidade



Conformidade



Invisibilidade

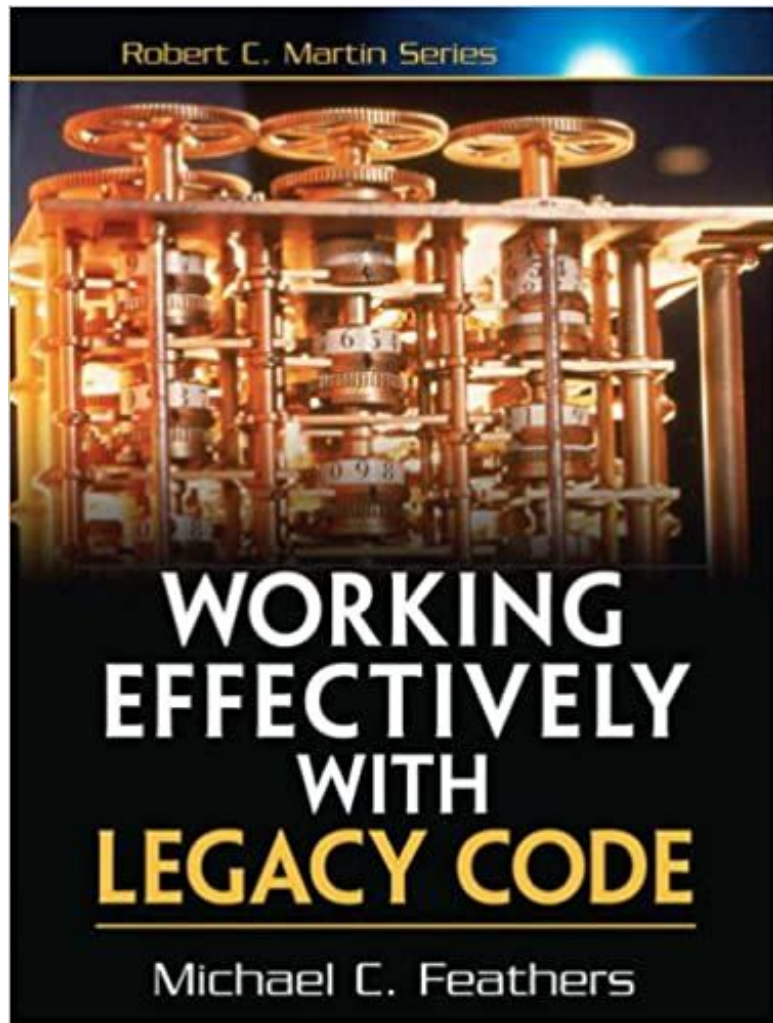


“A visão mais ingênua sobre o desenvolvimento de software ágil é que todo mundo chega e começa a escrever o código sem gastar um tempo inicial descobrindo o que fazer”

Martin Fowler, Chief Scientist da Thoughtworks

Teste

- V.t. Submeter a um teste; pôr à prova, experimentar.
- Prova que permite conhecer e avaliar as aptidões de um indivíduo, ou explorar-lhe a personalidade.
- Prova, **verificação** da eficiência ou do bom funcionamento (de máquinas, materiais etc.).



*“Code without tests
is bad code.” –
Michael Feathers*



Teste de Software

É a investigação do software a fim de fornecer informações sobre sua qualidade em relação ao contexto em que ele deve operar. Isso inclui o processo de utilizar o produto para revelar eventuais falhas.

Envolve ações que vão do levantamento de requisitos até o monitoramento do teste propriamente dito.



Mas pra que testar?

Ponto de vista dos engenheiros de software

Qualidade

- A qualidade está relacionada ao fato seu produto atender, ou não, as necessidades de seu cliente, sejam elas implícitas ou explícitas. Os testes ajudam a garantir que o produto atendeu todas as especificações.



Mas pra que testar?

Ponto de vista dos engenheiros de software

- Segurança

- Hoje, a maioria dos sistemas desenvolvidos conta com algum tipo de sistema de segurança, seja para uma área restrita de um site ou para lidar com transações de informações sigilosas.
- Dependendo do projeto os testes de segurança podem ser considerados fundamentais, valendo de tudo para tentar “burlar” o sistema.

Mas pra que testar?

Ponto de vista dos engenheiros de software

- Confiabilidade
 - Neste caso, os testes são para medir o período máximo de tempo que o software permanece funcionando sem apresentar falhas.
 - Muitas vezes durante os testes podem ser encontradas soluções para aumentar a confiabilidade do sistema.

Mas pra que testar?

Ponto de vista dos engenheiros de software

- Negócio
 - Os testes podem gerar informações importantes para a gerência de uma empresa influenciando na decisão de liberar, ou não, o sistema desenvolvido.
 - Neste caso, a equipe deve estudar as falhas encontradas, e então criar estratégias para eliminá-las.

E qual a dificuldade em testar?

A Realidade do Ensino: Foco na Construção

- Nossa introdução é via programação
- 3 anos aprendendo a construir
- Existe a Maratona de Programação (popular)



A Consequência: O Teste Esquecido

- Ninguém nos ensina a testar
- Criamos uma “rejeição” ao teste
- Alguém conhece a maratona de teste? (Desconhecida)



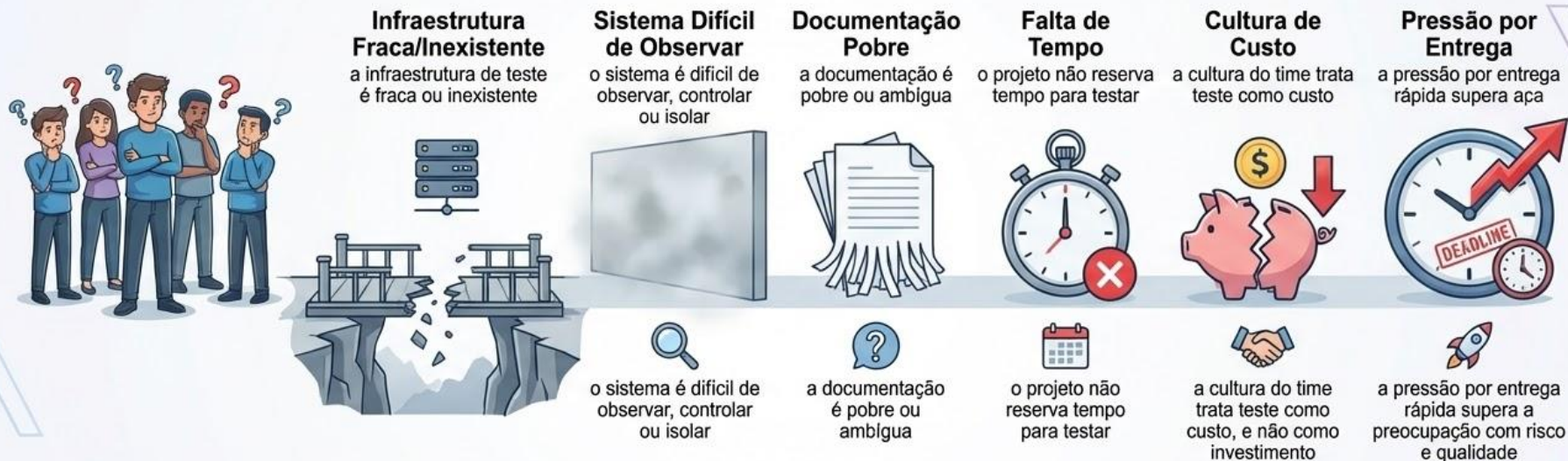
A dificuldade em testar é sociotécnica

Testar software não depende apenas de conhecer técnicas ou ferramentas. Na prática, o teste é condicionado por uma combinação de fatores:



Quando essas condições não favorecem o teste, mesmo equipes competentes podem testar pouco ou testar mal. O problema, portanto, não é apenas “falta de vontade”, mas o contexto em que o desenvolvimento acontece

Por que algumas equipes testam menos do que deveriam?



Isso mostra que o teste não falha apenas por deficiência técnica. Muitas vezes, ele falha porque o ambiente não “aceita” a prática de testar de modo eficiente e valioso para a equipe.



Primeira lição importante sobre teste

Se uma equipe não testa bem, a explicação não deve ser reduzida a “falta de competência”. É necessário perguntar:

Perguntas Essenciais:



O sistema é testável?



O time tem tempo e infraestrutura?



A cultura valoriza teste?



Os requisitos e comportamentos esperados estão claros?



Há apoio organizacional para manter a estratégia de qualidade?

Testar bem é uma capacidade técnica, mas também organizacional e social.



Teste X Depuração

Teste:

Processo de executar um programa ou sistema com o objetivo de revelar a presença de FALHAS; ou, falhando nesse objetivo, aumentar a confiança sobre o programa.

Depuração:

é uma consequência não previsível do teste. Após revelada a presença do erro, o defeito deve ser encontrado e corrigido

Depuração não é teste!



Objetivo dos testes de software

E software inteligente?
E IoT?

**Mostrar que o
produto não
está correto**



**Busca de um
contra-exemplo**



**Determinar entradas que
façam as saídas obtidas
diferirem das saídas
esperadas**

**Contra-exemplo: conjunto
de valores de entrada que
fazem com que o sistema
se comporte de forma
diferente do esperado**

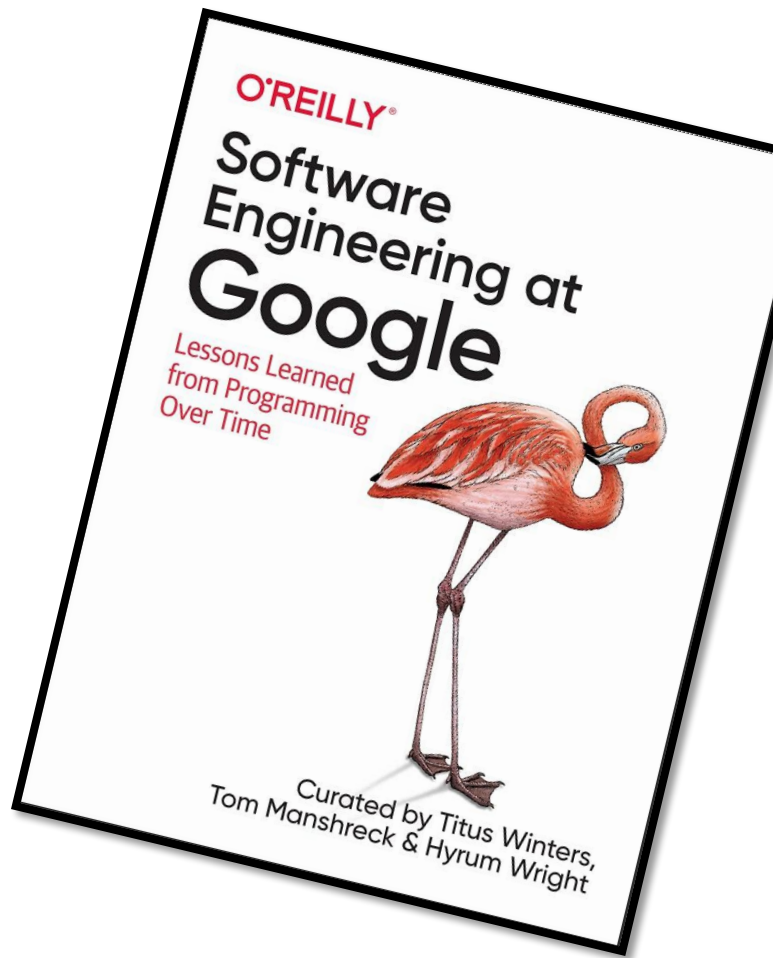
Alguns pontos de atenção

- Os testes devem ser rastreáveis aos requisitos do usuário
 - Tenho que saber quais testes avaliam cada requisito do sistema
- Devem ser planejados antes de sua execução
- Não é possível aplicá-los exaustivamente
- Testes com equipes independentes aumentam a eficácia



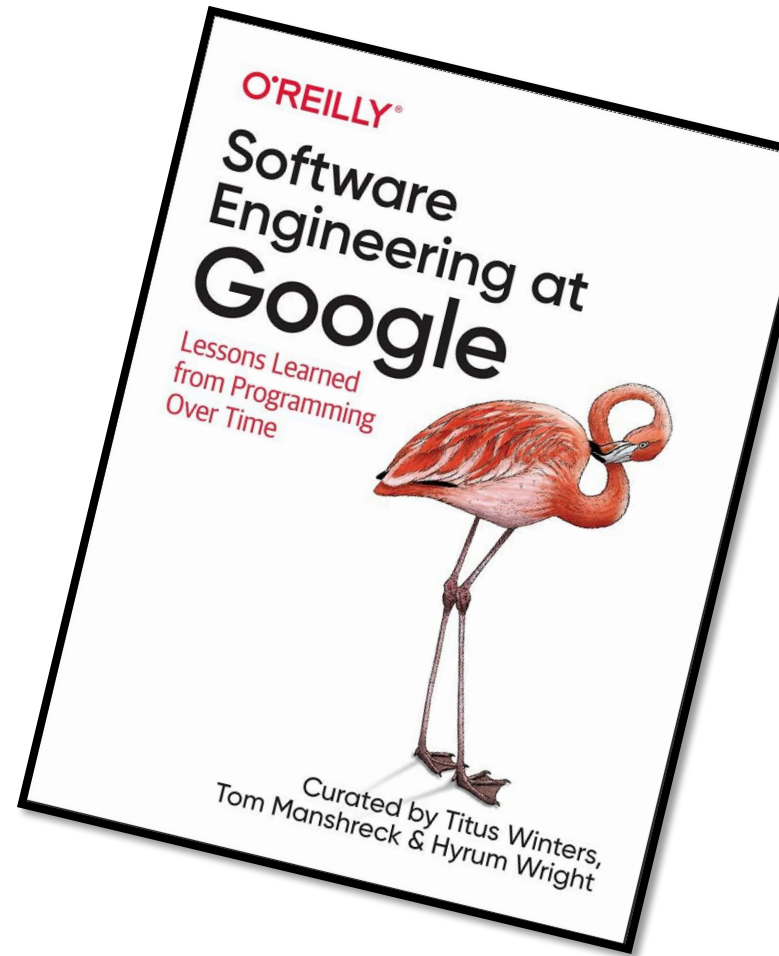
O teste mais simples...

- *Um único comportamento que está sendo testado, geralmente um método ou API que você está chamando*
- *Uma entrada específica, algum valor que você passa para a API*
- *Uma saída ou comportamento observável*
- *Um ambiente controlado, como um único processo isolado*



Benefícios de código de teste

- Para desenvolvedores vindos de organizações que não possuem uma forte cultura de testes, a ideia de escrever testes como meio de aumentar a produtividade e a velocidade pode parecer contraditória. Afinal, escrever testes pode levar tanto tempo (ou até mais!) quanto a implementação de uma funcionalidade.
- Pelo contrário, no Google, descobrimos que investir em testes de software traz vários benefícios essenciais para a produtividade dos desenvolvedores, como: menos depuração.



Escrever testes para um novo código é uma maneira prática de exercitar o design da API do próprio código.

Se um novo código for difícil de testar, isso geralmente ocorre porque ele possui muitas responsabilidades ou dependências difíceis de gerenciar.

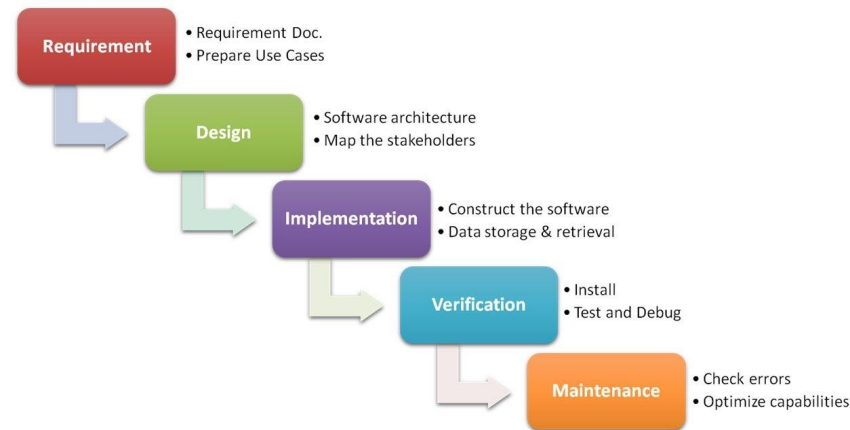
Um código bem projetado deve ser modular, evitando acoplamento excessivo e focando em responsabilidades específicas.



Um breve histórico

Quando o desenvolvimento era em cascata, os testes ocorriam em uma fase separada, após as fases de levantamento de requisitos, análise, projeto e codificação.

- Equipe separada de testes, responsável por verificar se a implementação atendia aos requisitos do sistema;
- Para checar isso, frequentemente os testes eram manuais;
 - Isto é, uma pessoa usava o sistema, informava dados de entrada e verificava se as saídas eram aquelas esperadas.
- Assim, o objetivo de tais testes era apenas detectar bugs, antes que o sistema entrasse em produção.



Com métodos ágeis...

- Parte dos testes passou a ser automatizada, isto é, além de implementar as classes de um sistema, os desenvolvedores passaram a implementar também código para testar tais classes. Assim, os programas tornaram-se auto-testáveis.
- Testes não são mais implementados após todas as classes de um sistema ficarem prontas. Eles são implementados até mesmo antes dessas classes.
- Não existem mais grandes equipes de testes ou elas são responsáveis por testes específicos. Em vez disso, o desenvolvedor que implementa uma classe também deve implementar os seus testes.
- Testes não são mais um instrumento exclusivo para detecção de bugs. Ganham novas funções, como verificar se uma classe continuará funcionando após um bug ser corrigido em uma outra parte do sistema. E testes são também usados como documentação para o código de produção.

How Developers Engineer Test Cases: An Observational Study

Maurício Aniche*, Christoph Treude†, Andy Zaidman*

*Delft University of Technology - The Netherlands

†University of Melbourne - Australia

m.f.aniche@tudelft.nl, christoph.treude@unimelb.edu.au, a.e.zaidman@tudelft.nl

Abstract—One of the main challenges that developers face when testing their systems lies in engineering test cases that are good enough to reveal bugs. And while our body of knowledge on software testing and automated test case generation is already quite significant, in practice, developers are still the ones responsible for engineering test cases manually. Therefore, understanding the developers' thought- and decision-making processes while engineering test cases is a fundamental step in making developers better at testing software. In this paper, we observe 13 developers thinking-aloud while testing different real-world open-source methods, and use these observations to explain how developers engineer test cases. We then challenge and augment our main findings by surveying 72 software developers on their testing practices. We discuss our results from three different angles. First, we propose a general framework that explains how developers reason about testing. Second, we propose and describe in detail the three different overarching strategies that developers apply when testing. Third, we compare and relate our observations with the existing body of knowledge and propose future studies that would advance our knowledge on the topic.

A Importância da Testabilidade

Boas práticas para escrita de testes - apesar dos avanços na geração automática de testes, os desenvolvedores ainda são os principais responsáveis por criar casos de teste eficazes.

O Papel do Desenvolvedor no Processo de Teste

Diferença entre testes de caixa-preta (baseados na especificação) e caixa-branca (baseados no código-fonte) - os desenvolvedores usam diferentes estratégias para criar testes, como testes guiados por código, por documentação ou por uma abordagem ad hoc.

A Construção de Modelos Mentais no Teste de Software

Leitura da documentação e do código-fonte para entender o comportamento esperado de um sistema - os desenvolvedores constroem um modelo mental da funcionalidade conforme testam.

O Uso de Cobertura de Código

Métricas de qualidade de testes, como cobertura de linhas, ramos e mutação - Muitos desenvolvedores usam ferramentas de cobertura para verificar se seus testes cobrem todas as partes do código.

Testes Além do “Happy Path”

Testes de valores-limite e de casos extremos - Os desenvolvedores não apenas testam cenários típicos, mas também exploram casos de borda e exceções.

A Influência da Documentação na Engenharia de Testes

Escrever especificações claras e garantir que requisitos estejam bem documentados - o estudo mostra que a qualidade da documentação impacta diretamente na criação de testes.

Refatoração de Código de Testes

Boas práticas de organização de testes, uso de fixtures, mocks e princípios como DRY (Don't Repeat Yourself) - desenvolvedores frequentemente reutilizam e refatoram código de testes.

O que tudo isso tem relação com o “aquecimento”?

Verificação: Antes de entregar, os alunos podem revisar se a lógica do código faz sentido e se a explicação está clara.

Validação: O colega que recebe a sequência tenta continuar. Se não conseguir, pode indicar um problema na definição ou ambiguidade, reforçando a importância da clareza em requisitos e testes.



Elementos do Teste de Software

- Caso de Teste:
 - Descreve um teste específico a ser executado sobre o software, sendo composto por valores de entrada, pré-condições ou restrições de execução, procedimentos e resultados esperados.
 - Exemplo:

Elementos do Teste de Software

CT01 – Autenticar usuário com credenciais válidas

Objetivo do teste:

Verificar se o sistema permite o acesso de um usuário que informa credenciais válidas.

Valores de entrada:

- E-mail: **ana@teste.com**
- Senha: **123456**

Pré-condições / restrições de execução:

- O usuário deve estar previamente cadastrado no sistema.
- A conta do usuário deve estar ativa.
- O sistema deve estar disponível e apresentar a tela de login.
- O usuário não deve estar autenticado no momento da execução do teste.

Procedimento:

1. Acessar a tela de login do sistema.
2. Informar o e-mail **ana@teste.com** no campo de e-mail.
3. Informar a senha **123456** no campo de senha.
4. Acionar o botão **Entrar**.

Resultado esperado:

- O sistema deve validar as credenciais informadas.
- O usuário deve ser autenticado com sucesso.
- O sistema deve redirecionar o usuário para a tela inicial.
- O nome ou identificador do usuário deve ser exibido na área logada do sistema.

Elementos do Teste de Software

- Roteiro de Teste:
 - Descreve os **passos** necessários para a execução de um ou grupos de casos de teste
 - Exemplo:

Passo	Ação	Caso de Teste
01	Testar venda com produto inexistente	CT-F001
02	Testar venda com produto indisponível no estoque	CT-F002
03	Testar venda válida	CT-S001

Elementos do Teste de Software

Critérios de cobertura de teste:

Estabelecem quais elementos do programa devem ser exercitados durante os testes, auxiliando na avaliação da abrangência e suficiência do teste.

Cobertura de teste:

Representa o percentual de elementos do programa que foi efetivamente exercitado pelos testes.

**Critério de cobertura:**

Define quais partes do sistema precisam ser exercitadas no teste.

Exemplo: no mapa Pokémon, o critério pode ser “visitar todas as áreas do mapa”.

Cobertura de teste:

Indica quantas dessas partes foram realmente exercitadas.

Exemplo: se o mapa tem 5 áreas e o teste visitou 4, a cobertura foi de **80%**.

Elementos do Teste de Software

- Rodada (Bateria) de Teste:
 - Execução de todos os testes para **uma versão do produto** em um determinado ambiente
 - Uma nova rodada de teste deve ser executada caso o critério de aceitação do produto não tenha sido atingido

Elementos do Teste de Software

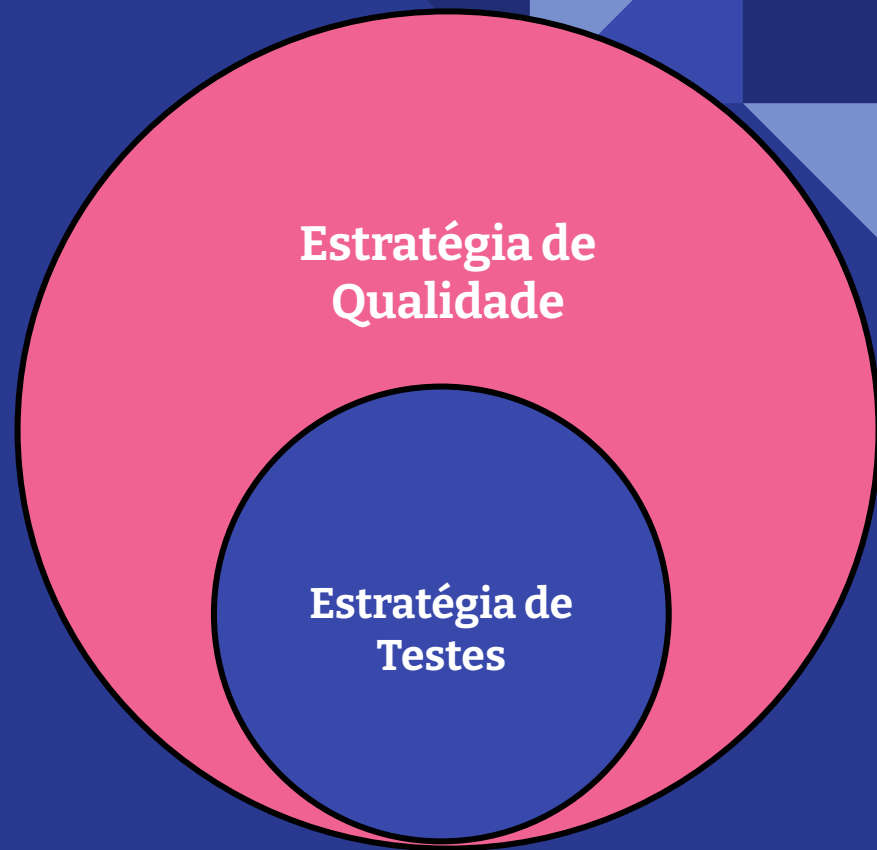
- Incidentes de Teste (*Issue* ou *Bug*):
 - **Evento ou comportamento diferenciado** ocorrido durante a execução dos testes que requer investigação
 - Não há garantia que todo incidente seja uma falha, pois ainda precisa ser analisado

Objetivo dos Testes de Software

- Mostrar que o produto não está correto!!
 - Determinar entradas que façam as saídas obtidas se desviar das saídas esperadas de acordo com a especificação
 - Busca de um contra-exemplo
 - Contra-exemplo: conjunto de valores de entrada que fazem com que o sistema se comporte de forma diferente do esperado.

Estratégia de testes

Próximo dia...





“Em particular, as ferramentas de análise estática são incapazes de lhe dar confiança em sua lógica de negócios.

Os testes de unidade são incapazes de garantir que, quando você chama uma dependência, está chamando-a adequadamente.

Os testes de integração de interface do usuário são incapazes de garantir que você esteja passando os dados corretos para seu back-end e que você responda e analise os erros corretamente.

Testes de ponta a ponta são bastante capazes, mas normalmente você os executa em um ambiente de não produção ”

Kent C. Dodds

Toda estratégia
conta uma história.

Qual vai ser a sua?

DANA NORRIS



STORYTELLING NA PRÁTICA

10 REGRAS SIMPLES PARA
CONTAR UMA BOA HISTÓRIA

ubook